

LadderSort: Adaptive Sorting via Interleaved Nondecreasing Subsequences

MALIK M. H. NAYAB¹, AUTHOR 2², AND AUTHOR 3³

¹COMSATS University Islamabad, Abbottabad Campus, Pakistan (e-mail: hamza.nayab48@gmail.com)

²COMSATS University Islamabad, Abbottabad Campus, Pakistan (e-mail: author2@cuiatd.edu.pk)

³COMSATS University Islamabad, Abbottabad Campus, Pakistan (e-mail: author3@cuiatd.edu.pk)

Corresponding author: Malik M. H. Nayab (e-mail: hamza.nayab48@gmail.com).

ABSTRACT Adaptive sorting algorithms take benefit of pre-existing structures in input to achieve performance better than $\Theta(N \log N)$. Traditional methods, like TimSort, estimate the presortedness measure based on contiguous runs. However, they prove to be less effective when input is based on interleaved sorted sequences. This paper presents LadderSort, a deterministic comparison-based sorting algorithm that adapts to K , the minimum number of nondecreasing subsequences covering the input, which equals both the offline optimum $K^*(A)$ and the length of the Longest strictly Decreasing Subsequence $\text{LDS}(A)$. The algorithm operates in two phases: Phase 1 decomposes the input into K nondecreasing ladders via a hinted greedy scan, and Phase 2 merges them using a K -adaptive strategy (galloping two-way merge for $K = 2$, loser-tree K -way merge for $K > 2$). LadderSort runs in $O(N \log(K + 1))$ time and $O(N + K)$ auxiliary space, interpolating between linear time on sorted input and $O(N \log N)$ on adversarial input. Experiments on eight synthetic workloads and a real GH Archive trace ($K = 368$) confirm practical speedups over TimSort on small- K interleaved inputs while preserving graceful degradation on adversarial inputs. We observe that LadderSort achieves faster execution time of up to 1.45 times over TimSort in wall-clock time on interleaved-stream workloads, particularly when $K \ll N$, while remaining competitive on random inputs, with a hybrid fallback variant ensuring graceful degradation on adversarial reverse-sorted inputs.

INDEX TERMS adaptive sorting, presortedness measures, nondecreasing subsequences, multiway merge, loser tree, streaming data

I. INTRODUCTION

SORTING is among the most fundamental primitives in algorithm design. It underpins database query processing, search-index construction, stream-join operators, analytics pipelines, and event-driven architectures. Classical comparison-based algorithms achieve $\Theta(N \log N)$ worst-case time, which is information-theoretically optimal for arbitrary inputs. However, data encountered in practice is rarely arbitrary. In many modern systems, the input to a sort routine arrives not as a random permutation but as a *fan-in* of several locally ordered producer streams, such as event logs from distributed services, timestamped telemetry from a fleet of devices, social-media timelines merged for display, posting-list segments in search-index maintenance, venue-specific feeds in market-data processing, or partition outputs in data-pipeline fan-in stages. Each producer emits a nondecreasing stream, and the aggregate sequence that reaches the sorting layer is a bursty, noncontiguous interleaving of those streams. Such sequences carry substantial *latent order* that a suffi-

ciently adaptive algorithm can exploit to sort well below the $N \log N$ barrier.

To understand why run-adaptive sorting is not enough, consider the dominant family of adaptive sorting algorithms. These algorithms, most notably TimSort [1] and PowerSort [2], measure the exploitable order in a sequence A by $\text{Runs}(A)$, the number of maximal contiguous monotone blocks (“runs”). A contiguous run is only useful when consecutive sorted elements appear next to each other in memory. In the fan-in workloads, each producer stream is internally sorted, but records from different producers are interleaved in the aggregate buffer. The interleaving process fragments each stream into many short contiguous runs, such that every switch from one producer to another can start a new run. When context switches are frequent (e.g., the common case in bursty telemetry, round-robin feed merging, or consolidated market feeds), the run count satisfies $\text{Runs}(A) = \Theta(N)$, even though, the sequence is an interleaving of only $K \ll N$ nondecreasing subsequences. Because the cost of run-adaptive

algorithms is $O(N + N \log \text{Runs}(A))$, they are forced into $\Theta(N \log N)$ behavior on precisely the workloads that carry the most exploitable structure. A sorting algorithm that measures cost by the number of interleaved sorted subsequences K rather than by $\text{Runs}(A)$ can sort in $O(N \log(K + 1))$ time, equivalently $O(N \log K)$ for $K \geq 2$. To understand the noncontiguous order problem, suppose three devices produce timestamped records:

$$D_1 : 3, 7, 8 \quad D_2 : 2, 5 \quad D_3 : 4.$$

An arrival buffer that interleaves these streams might contain $A = \langle 3, 7, 2, 5, 4, 8 \rangle$. A run-adaptive algorithm performs several short contiguous runs ($\langle 3, 7 \rangle$, then a drop to 2, and so on), yielding a high run count relative to N . Yet the sequence is naturally explained as three nondecreasing subsequences (ladders) that can be recovered by a single left-to-right scan: $L_1 = \langle 3, 7, 8 \rangle$, $L_2 = \langle 2, 5 \rangle$, $L_3 = \langle 4 \rangle$. After the identification of ladders, the sorted output is produced by a 3-way merge mechanism.

The aforementioned example highlights a major limitation of adaptive run algorithms. Although A is generated from three initially sorted streams ($K = 3$), run-based algorithms perceive a high degree of disorder because the sorted elements are not contiguous in memory. As a result, they fail to exploit the interleaved presortedness within the input and fall back to $O(N \log N)$ performance. This limitation motivates the need for a sorting algorithm that adapts to the interleaved presortedness of the structure instead of relying on $\text{Runs}(A)$. To address this gap, this paper presents LadderSort, a comparison-based sorting algorithm that decomposes the input into small, nondecreasing ladders, which are then merged adaptively. The LadderSort algorithm operates in two phases:

Phase 1 (decomposition): In this phase, the input A is traversed from left to right. Each key x is appended to the leftmost ladder whose current tail is at most x . In case, no such ladder exists, a new ladder is created. The resulting decomposition produces K nondecreasing ladders.

Phase 2 (adaptive merge): This phase merges the K ladders using a strategy depending upon value of K : direct return for $K = 1$, a galloping two-way merge for $K = 2$, and a loser-tree K -way merge for $K > 2$. The combined cost is $O(N \log(K + 1))$ time, equivalently $O(N \log K)$ for $K \geq 2$, and $O(N + K)$ auxiliary space.

The parameter K is not arbitrarily selected. Under the algorithm's placement rule, the online ladder count $K_{\text{LS}}(A)$ equals the offline optimum $K^*(A)$ (the minimum number of nondecreasing subsequences covering A), which in turn equals the length of the Longest strictly Decreasing Subsequence, i.e., $\text{LDS}(A)$. That is, $K_{\text{LS}}(A) = K^*(A) = \text{LDS}(A)$; the formal proof appears in Theorem II.1. LadderSort therefore performs best on inputs with small K but many contiguous runs, particularly in interleaved-stream workloads where run-adaptive methods perform poorly. The following is the summary of our contributions.

- 1) We propose LadderSort, a comparison-based sorting algorithm that exploits the presortedness measure K and

achieves $O(N \log(K + 1))$ running time using $O(N + K)$ auxiliary space.

- 2) Theoretical analysis of LadderSort is presented including correctness proofs, optimality of its ladder decomposition ($K_{\text{LS}}(A) = K^*(A) = \text{LDS}(A)$), and time, space, and average-case complexity analyses.
- 3) We show that the presortedness measure K complements classical measures such as Runs , capturing noncontiguous order that run-based measures cannot represent.
- 4) A hybrid variant of LadderSort is introduced that detects large- K inputs and falls back to a conventional $O(N \log N)$ sorting algorithm to ensure robust worst-case performance.
- 5) Extensive experiments are conducted on synthetic and real-world datasets that depict that LadderSort is effective on small- K interleaved-stream inputs, while it maintains graceful performance on adversarial workloads.

The paper is organized as follows. The preliminaries and notations are discussed in Section II. Literature review on adaptive sorting and presortedness measures is presented in Section III. The LadderSort algorithm is explained in detail in Section IV, including its two phases and the underlying placement rule. A theoretical analysis and proof of correctness, optimality, and time & space complexity of LadderSort is presented in Section V. Performance evaluation results of LadderSort are presented in Section VI on synthetic and real-world datasets. Limitations and tradeoffs of the proposed algorithm are discussed in Section VII. Finally, Section VIII concludes the paper and outlines directions for future research.

II. BACKGROUND AND PRELIMINARIES

This section presents the notations and definitions used throughout the paper. The specific terminologies employed in the discussions of existing literature in Section 3 (Related Work III) are also defined here. We assume a totally ordered domain $(\mathcal{D}, \leq)$ from which a sequence of N keys $A = \langle a_1, a_2, \dots, a_N \rangle$ are drawn. We denote the set of integers from 1 to m as $\{1, 2, \dots, m\}$. The goal is to compute a nondecreasing permutation S of A , where “sorted” means nondecreasing under the order $\leq$. In the rest of the paper, we use $\log$ to denote $\log_2$. Table 1 summarizes the primary mathematical notations.

TABLE 1. Summary of mathematical notation.

Notation	Description
A, N	Input sequence $A = \langle a_1, \dots, a_N \rangle$ and its length N
K	Dynamic number of ladders (nondecreasing subsequences) produced by the algorithm
$K_{LS}(A)$	Formal function mapping A to the ladder count produced by Phase 1
$K^*(A)$	Offline optimum: minimum number of nondecreasing subsequences covering A
$LDS(A)$	Length of the longest strictly decreasing subsequence of A
$\text{tops}[1..K]$	Contiguous array maintaining the tail (maximum) element of each active ladder
$\pi(x; \text{tops})$	Placement index: leftmost ladder j such that $\text{tops}[j] \leq x$
$\oplus$	Sequence concatenation operator (used in proofs)
$\prec$	Total order for deterministic tie-breaking by ladder identifier during the merge

Definition II.1 (Classical presortedness measures).

$\text{Runs}(A) :=$ number of maximal nondecreasing contiguous blocks in A .

$$\text{Inv}(A) := |\{(i, j) : i < j, a_i > a_j\}|. \quad (2)$$

$\text{Rem}(A) := N - \text{length of longest nondecreasing subsequence of } A$.

$\text{Exc}(A) :=$ minimum number of exchanges/transpositions to sort A .

$\text{SortedSeq}(A) :=$ minimum number of nondecreasing subsequences whose union covers A .

where $j, \ell \in [K]$ are ladder identifiers assigned in creation order. This makes the merge deterministic on duplicate keys. Note that the LadderSort is not globally stable.

Proposition II.1 (Dilworth equality for sequences). *For any sequence A , let $K^*(A) := \text{SortedSeq}(A)$ be the minimum number of nondecreasing subsequences that cover A , and let $LDS(A)$ be the length of a longest strictly decreasing subsequence of A . Then*

$$K^*(A) = LDS(A).$$

The offline minimum $K^*(A)$ can be computed in $O(N \log N)$ time using standard patience-sorting routines [3], [4]. The following Theorem II.1 shows that the online greedy placement used in Phase 1 (Section IV-B) achieves this offline optimum under our placement/tie convention.

Theorem II.1 (Optimality of the greedy ladder decomposition). *For every input sequence A ,*

$$K_{LS}(A) = K^*(A) = LDS(A).$$

Any strictly decreasing subsequence of length r requires r distinct nondecreasing subsequences in any cover, so $LDS(A) \leq K^*(A)$. By Dilworth's Theorem (Proposition II.1), $K^*(A) = LDS(A)$. Combining these inequalities:

$$LDS(A) \geq K_{LS}(A), \quad K^*(A) \leq K_{LS}(A), \quad K^*(A) = LDS(A).$$

Therefore, $K_{LS}(A) = K^*(A) = LDS(A)$.

Definition II.2 (Ladder). A ladder is a finite sequence $L = \langle \ell_1, \dots, \ell_m \rangle$ such that $\ell_1 \leq \ell_2 \leq \dots \leq \ell_m$.

Definition II.3 (Tail and tail array). For a nonempty ladder L , its *tail* is defined as $\text{tail}(L) := \ell_m$. Given an ordered collection of K nonempty ladders $\mathcal{L} = (L_1, L_2, \dots, L_K)$, the corresponding *tail array* $\text{tops}[1..K]$ is a contiguous array where $\text{tops}[j] := \text{tail}(L_j)$ for all $j \in [K]$. The collection $\mathcal{L}$ is said to satisfy the *tail monotonicity property* if:

$$\text{tops}[1] \geq \text{tops}[2] \geq \dots \geq \text{tops}[K].$$

Definition II.4 (Placement index). For a key x and a nonincreasing array $\text{tops}[1..K]$, the *placement index* is

$$\pi(x; \text{tops}) := \min\{j \in [K] : \text{tops}[j] \leq x\},$$

taking $\pi(x; \text{tops}) := K + 1$ if no such j exists.

Definition II.5 (Greedy ladder count K_{LS}). Let $K_{LS}(A)$ denote the total number of ladders constructed by sequentially processing the sequence A from left to right under the online greedy placement rule specified in Definition II.4. Alternatively, $K_{LS}(A) = |\text{tops}|$ upon consuming the final element of A . We reserve $K^*(A) := \text{SortedSeq}(A)$ to denote the absolute offline optimal cover (Definition II.1).

Definition II.6 (Run-identifier tie-break). During K -way merge, comparisons use the total order:

$$(x, j) \prec (y, \ell) \iff (x < y) \vee (x = y \wedge j < \ell),$$

III. RELATED WORK

In this section, we review the literature on adaptive sorting and presortedness measures, and compare LadderSort in the context of existing frameworks. We highlight the limitations of run-adaptive methods on interleaved-stream workloads and motivate the need for a measure that captures noncontiguous order.

A. EXISTING PRESORTEDNESS FRAMEWORKS

Adaptive sorting theory formalizes algorithmic cost as a function of a *measure of presortedness*: a mapping m from a sequence A of length n to a non-negative integer satisfying standard monotonicity and normalization axioms [5]. Maniila's foundational framework establishes that a measure m is meaningful only if the execution cost of a sorting algorithm can be bounded as a function of both n and $m(A)$, rather than n alone [5]. Notably, it is not required by the framework that an algorithm optimal with respect to m_1 be optimal with respect to m_2 if the two measures are not mutually bounded; different measures reflect different aspects of the input's structural properties.

Estivill-Castro and Wood [6] study the presortedness measures of classical type, which includes the number of contiguous runs $\text{Runs}(A)$, the number of inversions $\text{Inv}(A)$, the removal distance $\text{Rem}(A)$, and the exchange distance $\text{Exc}(A)$ (see Section II, Definition II.1). These actions have been partially ranked according to their ability to discriminate.

For example, for all sequences A ; $\text{Inv}(A) \geq \text{Runs}(A) - 1$; therefore an algorithm optimal under Inv is automatically as adaptive as an algorithm optimal under Runs and the other way around is not true.

The measure $\text{SortedSeq}(A)$, which is the number of minimal nondecreasing subsequences needed to cover A is a unique entity in this hierarchy. A duality between covering and extremal sequences, between $\text{SortedSeq}(A)$ and the longest strictly decreasing subsequence of A is given by the celebrated Dilworth's Theorem [7]. LadderSort is designed specifically to address this structural property, with its K parameter. In particular we prove in Theorem II.1 that the online ladder count equals $\text{SortedSeq}(A) = \text{LDS}(A)$.

Numerous existing works explore *contiguous* monotone runs. An example is *natural MergeSort* that analyzes existing runs and sequentially merges them [8]. Its successor *TimSort* [1] checks for ascending or descending runs, and utilizes binary insertion sort to enforce minimum run lengths. Using strict stack invariants, the *TimSort* manages the merge sequence to guarantee an $O(N + N \log \text{Runs}(A))$ time complexity. Alternatively, *PowerSort* [2] constructs near optimal merge trees with minimal overhead, further refined by multiway extension [9].

In [10], the monotone subsequences are created by greedily extending lists from both ends. This yields a presortedness measure capturing a broader structural order than $\text{Runs}(A)$. The classical hierarchy has tightly bound the number of *encroaching lists* between $\text{SortedSeq}(A)$ and $\text{Runs}(A)$. Similarly, the classical online greedy procedure, *patience sorting* [4] has pile count corresponding directly to extremal subsequence lengths. It tracks the longest increasing subsequence $\text{LIS}(A)$ or decreasing subsequence $\text{LDS}(A)$ depending upon specific convention used for tie-breaking convention.

The general purpose algorithms (Introsort [11], [12], BlockQuicksort [13] and Smoothsort [14]) have $O(N \log N)$ worst cases, but they are not aware of the structure of the input, and they do not take advantage of the presortedness. Tournament-like structures work very well with input sequences of multiway ordered streams. The *loser-tree* variant is also of major importance in K -way external-memory merging pipelines [8]. Initially, a loser tree with K leaves can be initialized in $O(K)$ time; each extract-minimum-and-advance step can be performed in $O(\log K)$ time. The loser of each match is stored at the internal nodes, which reduces the number of memory accesses yielding $\lceil \log K \rceil$ for each step as opposed to $2\lceil \log K \rceil$ for a typical winner tree. Hence, this architecture is commonly used in external sorting scenarios [15], [16] for managing stream of large volumes of data.

B. THE INTERLEAVING GAP

In all the run-adaptive methods which have been surveyed above (*TimSort*, *PowerSort*, *AdaptiveShiversSort*, and *natural mergesort*), the cost is a function of $\text{Runs}(A)$ representing the number of *contiguous* maximal monotone blocks. Such measure is suitable for inputs arriving as a few long sorted segments (e.g., log rotations, append-dominated tables, or

nearly sorted files), but it is unable to tell that the input is *noncontiguously* ordered. Considering K internally sorted producer streams that are interleaved and multiplexed into a single buffer in arrival order, with each producer contributing approximately N/K elements, every context switch from one producer to another may break a contiguous run. Usually, a sequence is an interleaving of only K nondecreasing subsequences, however, when the number of context switches is high (the usual situation for bursty telemetry, round robin feed merging or consolidated feed from the financial markets), the number of contiguous runs goes to $\text{Runs}(A) = \Theta(N)$. In such case, run adaptive sorts exhibit worst-case behavior on the workloads that have the most exploitable structure.

The above-mentioned performance gap stems from a fundamental mismatch between the actual structural parameter of the input sequence and the measure used by the algorithm. The appropriate parameter to exploit is the global optimal cover, $K = \text{SortedSeq}(A)$, representing the minimum number of nondecreasing subsequences spanning A . In the interleaved-stream scenario, $K \ll N$ and $K \ll \text{Runs}(A)$; consequently, an algorithm that adapts to K instead of $\text{Runs}(A)$ can achieve a time complexity of $O(N \log(K + 1))$. For any $K \geq 2$, this collapses to an optimal $O(N \log K)$ bound, offering massive performance gains over the oblivious $O(N \log N)$ worst-case bound.

Encroaching lists proposed in [10] partially mitigate the inability of run-adaptive sorting algorithms to recognize non-contiguous order structure. They primarily function as online partitioning mechanism to perform recognition of non-contiguous subsequence structures, but lack a deterministic merge policy that guarantees a strict $O(N \log(K + 1))$ execution bound. Alternatively, Aldous et al. [4] proposed patience sorting that performs an online sequence partition optimal count, but it lacked an integrated K -adaptive merge phase to synthesize the sorted output. Therefore, it remains an open challenge to bridge the gap between online noncontiguous partitioning and optimal multiway merging.

To consider the proposed K in the context of existing presortedness measures, we assume that $K_{\text{LS}}(A)$ represents the number of the ladders created by the online greedy placement rule of LadderSort, i.e., Definition II.4. This ladder count not only equals the offline optimal decomposition $K^*(A)$, but also equals the length of longest decreasing subsequence:

$$K_{\text{LS}}(A) = K^*(A) = \text{LDS}(A) \leq \text{Runs}(A).$$

The above inequality indicates that instead of capturing only the contiguous runs, these ladders also capture the globally ordered subsequences. For instance, if the input is formed by interleaving K_0 nondecreasing subsequences, then we have $K_{\text{LS}}(A) \leq K_0$. The equality holds when a decreasing subsequence of length K_0 exists in the interleaving. Consequently, $\text{Runs}(A)$ can be $\Theta(N)$, keeping the ladder decomposition optimal. This shows that K can capture the different forms of input order that are not detected by run-based measures.

The proposed measure is also related to inversion-based presortedness. For certain families of input sequences, $\text{Inv}(A) \geq \binom{K}{2}$, implying the asymptotic bound $K = O(\sqrt{\text{Inv}(A)})$. Thus, although inversion count and ladder count are related, they characterize different aspects of existing order.

C. OFFLINE SEQUENCE DECOMPOSITION

Dilworth's theorem gives a clean characterization of optimal sequence decomposition. The minimum number of nondecreasing subsequences needed to cover a sequence equals the length of its longest strictly decreasing subsequence [7]. In the current context, this implies that:

$$K^*(A) = \text{LDS}(A),$$

where $K^*(A)$ denotes the optimal offline ladder count which can be computed in $O(N \log N)$ time using patience-sorting-based routines [3], [4]. Such methods treat decomposition as an offline objective or as a presortedness measure, and not as the preliminary stage of a sorting algorithm.

In LadderSort, the ladders are constructed through an online greedy placement algorithm during input parsing, and merged to generate the sorted output. As will be shown in Section V, this online decomposition process yields the optimal count consistent with Dilworth's theorem, while adhering to a complete K -adaptive sorting framework.

IV. THE LADDERSORT ALGORITHM

We present the LadderSort algorithm including its both phases and basic placement rule. The algorithm is designed to take benefit of presortedness measure K , achieving optimal performance on interleaved-stream workloads, and show graceful performance degradation on adversarial inputs. Moreover, the LadderSort utilizes the three-way integrated approach to reduce the interleaving gap discussed in Section III-B.

A. OVERVIEW

LadderSort operates in two distinct phases (conceptually illustrated in Figure 1):

B. PHASE 1: SEQUENTIAL DECOMPOSITION

Phase 1 decomposes the input sequence A into an organized collection of non-decreasing ladders, represented as $\mathcal{L} = (L_1, L_2, \dots, L_K)$. The input is scanned from left to right and using an online greedy placement rule, places the incoming key to left most feasible ladder (using Definition II.4). Moreover, it keeps the tail array tops into a strictly decreasing order (Definition II.3). The aforementioned strategy limits the search problem to find the smallest index j where $\text{tops}[j] \leq x$.

The proposed design is inspired by the classical patience-sorting technique [4]. By combining sorted tail vector with monotonic search predicate, the online rule absolutely achieves $K_{\text{LS}}(A) = \text{SortedSeq}(A) = \text{LDS}(A)$ ladders

Algorithm 1 LadderSort(A)

Require: Input sequence $A = \langle a_1, \dots, a_N \rangle$

Ensure: Sorted sequence S

```

1: Initialize empty ladder list  $(L_1, \dots, L_K)$ ,  $K \leftarrow 0$ 
2: Initialize empty array  $\text{tops}$ 
3:  $\text{lastIdx} \leftarrow 1$ 
4: for  $i = 1$  to  $N$  do
5:    $x \leftarrow a_i$ 
6:    $j \leftarrow \text{HintedLowerBound}(\text{tops}, x, \text{lastIdx})$ 
7:   if  $j = K + 1$  then
8:      $K \leftarrow K + 1$ ;  $L_K \leftarrow \langle x \rangle$ ;  $\text{tops}[K] \leftarrow x$ 
9:   else
10:    Append  $x$  to  $L_j$ ;  $\text{tops}[j] \leftarrow x$ 
11:   end if
12:    $\text{lastIdx} \leftarrow j$ 
13: end for
14: return MergeByK( $L_1, \dots, L_K$ )
```

(Theorem II.1). In previous works, such decompositions are considered as static offline measures. On the contrary, the LadderSort utilizes this feature as a stateful first stage prior to optimal multi-way merge.

Algorithm 1 provides an overview of the whole process. All routing decisions are handed over to HintedLowerBound (Algorithm 2) by the main routine. This subroutine utilizes three-phase locality-aware search to find insertion index in worst case $O(\log K)$.

1) Sequential Decomposition Mechanics

The main decomposition loop processes the input in a single pass and tracks active tail of each ladder in tops vector. The algorithm reduces the lookup overhead for those consecutive elements that arrive from the same stream producer. It accomplishes this by caching the latest modified insertion position (lastIdx) and passing it to the search subroutine. This stateful tracking measure effectively maintains a communication between pipeline's macroscopic fan-in structure and microscopic element placement logic, and ensures to take full advantage of temporal locality.

2) Stateful Hinting and Memory Layout

The placement search over the tail array is accelerated via a stateful three-stage procedure designed to exploit the temporal patterns of interleaved fan-in streams:

- 1) **Stage 1 (Local Probe):** It checks whether the current element can be appended with the ladder that was updated in the previous iteration ($j = \text{lastIdx}$). This helps in achieving $O(1)$ processing when consecutive tokens are arriving from the same producer.
- 2) **Stage 2 (Exponential Gallop):** If the local probe fails, this stage passes a search interval by exponentially doubling the stride parameter at the right of hint parameter till a valid boundary or end of array is encountered.
- 3) **Stage 3 (Binary Search):**

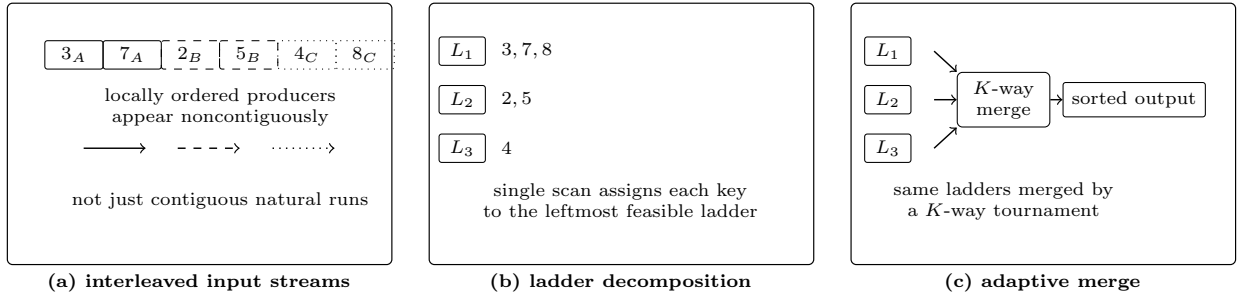


FIGURE 1. LadderSort targets input profiles that present as interleavings of locally ordered data streams. Phase 1 decomposes the sequence into nondecreasing ladders via an online leftmost-feasible placement rule; in this trace, $L_1 = (3, 7, 8)$, $L_2 = (2, 5)$, and $L_3 = (4)$. Phase 2 merges these subsystems adaptively, guaranteeing an optimal overall complexity of $O(N \log(K + 1))$.

This stage finds the exact insertion position in the bounded interval and guarantees a strict worst-case cost $O(\log K)$ for each element. It helps to maintain $O(1)$ performance in normal high-locality workloads.

The array `tops[1..K]` is physically allocated as a contiguous memory vector. For practical workload dimensions where $K \ll \text{Runs}(A)$, this structure remains highly compact and cache-resident, making the localized search sweeps exceptionally cache-friendly. We purposely refrain from making specific assertions regarding exact hardware performance metrics (such as target cache lines, translation lookaside buffer hit rates, or speculative branch behaviors) as these dynamics are heavily dependent on target execution environments and architectural specifications.

C. PHASE 2: ADAPTIVE MERGE ARCHITECTURE

Phase 2 converts the sorted nondecreasing ladders $\mathcal{L}$ into a single globally ordered output sequence S . Execution paths are selected dynamically based on actual ladder count K . A direct return is made for $K = 1$, optimized galloping two-way merge (Algorithm 4) is performed for $K = 2$, and for $K > 2$, the deterministic K -way loser-tree merge (Algorithm 5) is utilized. When multi-way streams ($K > 2$) are processed, then for each extracted element, the tournament-style loser tree performs an upward update path with a maximum of $\lceil \log K \rceil$ comparisons. The ties are deterministically broken using ladder identifier (Definition II.6) that results in reproducible output. The combined complexity of sequence decomposition phase (Phase 1) and adaptive merging (Phase 2) comes up to $O(N \log(K + 1))$, while auxiliary workspace requires $O(N + K)$.

1) Adaptive Flow Control

Multi-path control logic plays an important role as a low-overhead algorithmic guardrail. It is designed to counter execution penalties that occur during processing of those inputs whose structures are highly localized. When $K = 1$, the system completely bypasses the merging framework and returns the single pre-sorted ladder. This guarantees a strict $O(N)$ linear-time execution on pre-ordered inputs. When $K = 2$, the execution branches into a specialized two-

way galloping loop. This reduces the structural overhead in initializing and updating the internal tournament structure. Generalized multi-way tournament tree is reserved for those situations where $K > 2$. In this way, the initialization cost and tree traversal penalties occur only when dataset's fan-in complexity necessitates them.

2) Galloping Merge Strategy

To take benefit of long-range monotonic runs in the generated ladders, galloping merge loop is utilized only for $K = 2$. Instead of comparing with each element during merge procedure, the algorithm uses binary search following exponential search so that a specific contiguous block can be recognized which can be safely emitted [17]. When the two ladders alternate in short turns, this becomes nearly similar to element-by-element comparisons. If a ladder dominates on some larger range, then block of length r can be identified in $O(\log r)$ comparisons, and then it is emitted with a single pointer update.

3) Loser-tree Memory Layout

For the general case ($K > 2$), the loser tree is represented as an implicit binary tree stored in a flat array of K integer entries, following the standard heap-array organization [8]. Entry $T[0]$ stores the current overall winner, while the remaining $K - 1$ entries record the losers of the internal tournament matches. An auxiliary array of K head pointers maintains the current front element of each ladder. After each output element is produced, only the path from the corresponding leaf to the root is replayed to restore the tournament, requiring a single upward traversal of at most $\lceil \log K \rceil$ internal nodes. This compact representation requires $O(K)$ auxiliary memory and is well suited to efficient tournament updates while avoiding unnecessary pointer-based structures.

V. THEORETICAL ANALYSIS

In this section, we analyze the correctness and complexity of LadderSort. We prove that the online greedy placement rule produces an optimal decomposition into K nondecreasing ladders, and that the subsequent merge phase produces a globally sorted output. We then analyze the time and space

Algorithm 2 HintedLowerBound($\text{tops}[1..K], x, \text{lastIdx}$)

Require: Nonincreasing tail vector $\text{tops}[1..K]$, target key x , historical hint $\text{lastIdx} \in [1, K]$

Ensure: First index $j \in [K]$ satisfying $\text{tops}[j] \leq x$, or $K+1$ if no such position exists

```

1: if  $K = 0$  then
2:   return 1      ▷ First element initializes the primary ladder
3: end if

4: Stage 1: Local Probe
5: if  $\text{tops}[\text{lastIdx}] \leq x$  then
6:   return  $\text{lastIdx}$       ▷ Element appends to the immediate prior ladder
7: end if

8: Stage 2: Exponential Gallop
9:  $s \leftarrow 1$ 
10: while  $\text{lastIdx} + s \leq K$  and  $\text{tops}[\text{lastIdx} + s] > x$  do
11:    $s \leftarrow 2 \cdot s$ 
12: end while
13:  $lo \leftarrow \text{lastIdx} + \lfloor s/2 \rfloor$ 
14:  $hi \leftarrow \min(K, \text{lastIdx} + s)$ 

15: Stage 3: Binary Search
16:  $j \leftarrow K + 1$ 
17: while  $lo \leq hi$  do
18:    $mid \leftarrow lo + \lfloor (hi - lo)/2 \rfloor$ 
19:   if  $\text{tops}[mid] \leq x$  then
20:      $j \leftarrow mid$ ;  $hi \leftarrow mid - 1$       ▷ Search left for smaller valid index
21:   else
22:      $lo \leftarrow mid + 1$ 
23:   end if
24: end while
25: return  $j$ 

```

complexity of both phases, showing that the overall algorithm runs in $O(N \log(K+1))$ time with $O(N+K)$ auxiliary space.

A. CORRECTNESS

Lemma V.1 (Each ladder remains sorted). *After processing any prefix $\langle a_1, \dots, a_t \rangle$ in Phase 1, every ladder L_j is nondecreasing.*

Proof. Induction on t . Base case $t = 1$: $L_1 = \langle a_1 \rangle$ is trivially nondecreasing. Inductive step: assume all ladders are nondecreasing after processing $a_1, \dots, a_{t-1}$. At step t , $x = a_t$ is assigned to ladder L_j where $j = \pi(x; \text{tops})$. If $j = K+1$, a new singleton ladder $\langle x \rangle$ is created, which is sorted. Otherwise, j is the first index with $\text{tops}[j] \leq x$, so $\text{tops}[j] = \text{tail}(L_j) \leq x$; appending x to L_j preserves the nondecreasing property. All other ladders are unchanged. $\square$

Algorithm 3 MergeByK($L_1, \dots, L_K$)

Require: Collection of nondecreasing ladders $\mathcal{L} = (L_1, \dots, L_K)$

Ensure: Globally sorted sequence S

```

1: if  $K = 0$  then
2:   return  $\langle \rangle$ 
3: end if
4: if  $K = 1$  then
5:   return  $L_1$ 
6: end if
7: if  $K = 2$  then
8:   return TwoWayGalloping( $L_1, L_2$ )
9: end if
10: return KWayLoserTree( $L_1, \dots, L_K$ )

```

Lemma V.2 (Strict tail invariant). *After each insertion step of Phase 1 the tail array satisfies the strict inequality*

$$\text{tops}[1] > \text{tops}[2] > \dots > \text{tops}[K].$$

In particular the array is nonincreasing and, in fact, strictly decreasing at all times.

Proof. We prove by induction on the number of processed elements that $\text{tops}[1] > \text{tops}[2] > \dots > \text{tops}[K]$ is maintained.

Base case. After the first insertion, $K = 1$ and the invariant is vacuously true.

Inductive step. Assume the invariant $\text{tops}[1] > \text{tops}[2] > \dots > \text{tops}[K]$ holds before inserting element x . Let $j = \pi(x; \text{tops})$ be the first feasible index.

Case $j = K+1$ (new ladder). Then $x < \text{tops}[p]$ for all $p \in [1..K]$, so $x < \text{tops}[K]$. We set $\text{tops}[K+1] := x$. By the induction hypothesis, the old sequence $\text{tops}[1] > \dots > \text{tops}[K]$ is strictly decreasing. Since $x < \text{tops}[K]$, the new sequence $\text{tops}[1] > \dots > \text{tops}[K] > \text{tops}[K+1]$ is also strictly decreasing.

Case $j \leq K$ (update existing ladder). We update $\text{tops}[j] := x$.

For $p < j$: by definition of the first-feasible rule, $\text{tops}[p] > x$. These values are unchanged.

For $p = j$: the new value is x .

For $p > j$: the values $\text{tops}[p]$ are unchanged. By the induction hypothesis, $\text{tops}[j] > \text{tops}[j+1]$ held before the update. Since the old $\text{tops}[j] \leq x$ (by feasibility), we have $\text{tops}[j+1] < \text{tops}[j] \leq x$, thus $\text{tops}[j+1] < x$. After the update, the new value at position j is x and at position $j+1$ is $\text{tops}[j+1]$, so $x > \text{tops}[j+1]$.

Boundary cases: If $j = 1$, there are no positions $p < 1$. If $j = K$, there are no positions $p > K$. Both cases are covered by the logic above.

Therefore, the updated sequence $\text{tops}[1] > \dots > \text{tops}[K]$ remains strictly decreasing. $\square$

Lemma V.3 (Merge outputs a globally sorted list). *Assume $L_1, \dots, L_K$ are nondecreasing. Then MergeByK($L_1, \dots, L_K$)*

Algorithm 4 TwoWayGalloping(L_1, L_2)**Require:** Sorted input ladders L_1 and L_2 **Ensure:** Sequentially merged sorted output sequence S

```

1:  $p \leftarrow 1$ ;  $q \leftarrow 1$ ;  $S \leftarrow \langle \rangle$ 
2: while  $p \leq |L_1|$  and  $q \leq |L_2|$  do
3:   if  $L_1[p] \leq L_2[q]$  then
4:      $s \leftarrow 1$   $\triangleright$  Exponential gallop rightward in  $L_1$ 
5:     while  $p + s \leq |L_1|$  and  $L_1[p + s] \leq L_2[q]$  do
6:        $s \leftarrow 2 \cdot s$ 
7:     end while
8:      $lo \leftarrow p + \lfloor s/2 \rfloor$ ;  $hi \leftarrow \min(|L_1|, p + s)$ 
9:      $r \leftarrow lo$   $\triangleright$  Binary search for the upper bound
    prefix
10:    while  $lo \leq hi$  do
11:       $mid \leftarrow lo + \lfloor (hi - lo)/2 \rfloor$ 
12:      if  $L_1[mid] \leq L_2[q]$  then
13:         $r \leftarrow mid$ ;  $lo \leftarrow mid + 1$ 
14:      else
15:         $hi \leftarrow mid - 1$ 
16:      end if
17:    end while
18:    Append prefix  $L_1[p..r]$  to  $S$ ;  $p \leftarrow r + 1$ 
19:  else
20:     $s \leftarrow 1$   $\triangleright$  Analogous exponential gallop rightward
    in  $L_2$ 
21:    while  $q + s \leq |L_2|$  and  $L_2[q + s] < L_1[p]$  do
22:       $s \leftarrow 2 \cdot s$ 
23:    end while
24:     $lo \leftarrow q + \lfloor s/2 \rfloor$ ;  $hi \leftarrow \min(|L_2|, q + s)$ 
25:     $r \leftarrow lo$ 
26:    while  $lo \leq hi$  do
27:       $mid \leftarrow lo + \lfloor (hi - lo)/2 \rfloor$ 
28:      if  $L_2[mid] < L_1[p]$  then
29:         $r \leftarrow mid$ ;  $lo \leftarrow mid + 1$ 
30:      else
31:         $hi \leftarrow mid - 1$ 
32:      end if
33:    end while
34:    Append prefix  $L_2[q..r]$  to  $S$ ;  $q \leftarrow r + 1$ 
35:  end if
36: end while
37: Append any remaining elements from  $L_1[p..|L_1|]$  or
 $L_2[q..|L_2|]$  directly to  $S$ 
38: return  $S$ 

```

outputs a nondecreasing sequence S that is a permutation of the multiset union $\bigcup_{j=1}^K L_j$.

Proof. $K \in \{0, 1\}$: Trivial.

$K = 2$: TwoWayGalloping is a standard merge (accelerated by galloping, which only extends contiguous blocks that are provably $\leq$ the opposing head). Galloping does not alter correctness, only the number of comparisons. The result is a sorted permutation of $L_1 \cup L_2$.

$K > 2$: KWayLoserTree maintains one current head per

Algorithm 5 KWayLoserTree($L_1, \dots, L_K$)**Require:** K nondecreasing sequences; comparison order $\prec$ (Definition II.6)**Ensure:** Globally sorted merge S

```

1: Allocate internal loser-tree array  $T[0..K - 1]$  (node 0 =
    overall winner)
2: Initialize: for  $j = 1$  to  $K$ ,  $head[j] \leftarrow$  first element of  $L_j$ 
3: Run  $K - 1$  comparisons to build initial tournament; record
    losers at each internal node
4: while any sequence non-empty do
5:    $j^* \leftarrow T[0]$   $\triangleright$  index of overall winner (minimum)
6:   Emit  $head[j^*]$ 
7:   Advance  $L_{j^*}$  by one element; update  $head[j^*]$  (or  $+\infty$ 
    if exhausted)
8:   Walk path from leaf  $j^*$  to root, comparing with stored
    losers and updating  $T$   $\triangleright \lceil \log K \rceil$  comparisons; each
    node stores loser, so path = single upward pass
9: end while
10: return  $S$ 

```

ladder and repeatedly outputs the minimum head under $\prec$ (Definition II.6). Since each L_j is nondecreasing, advancing any pointer never decreases that ladder's head. At each step, the loser tree correctly identifies the global minimum head (by the standard correctness argument for tournament trees [8]), outputs it, advances the corresponding ladder, and restores the tournament in $O(\log K)$ time. Because $\prec$ is a total order refining $\leq$, the output sequence is nondecreasing in key order. Every element is output exactly once, so S is a permutation of the union. $\square$

Theorem V.4 (Correctness of LadderSort). *For any input $A = \langle a_1, \dots, a_N \rangle$ over a totally ordered domain, LadderSort(A) outputs a sequence S that is (i) a permutation of A and (ii) nondecreasing under $\leq$.*

Proof. By Lemma V.1, each L_j produced in Phase 1 is nondecreasing. Every element a_i is appended to exactly one ladder (or starts a new one), so the multiset union of the ladders equals A . Lemma V.2 ensures the tail-array invariant is maintained throughout, guaranteeing that the placement predicate is applied correctly. By Lemma V.3, Phase 2 outputs a nondecreasing permutation of the union, which equals a nondecreasing permutation of A . The overall implementation control flow of the decomposition and merge phases is illustrated in the flowchart of Figure 2. $\square$

B. TIME COMPLEXITY

Lemma V.5 (Time complexity). *LadderSort runs in $O(N \log(K + 1))$ time. More precisely, the algorithm is linear for $K = 1$ and runs in $O(N \log K)$ for $K \geq 2$.*

Proof. Phase 1. Each of the N elements requires one call to HintedLowerBound. Stage 1 (local probe) costs $O(1)$. Stage 2 (exponential gallop) doubles the stride until it reaches a feasible index or overshoots K ; the number of doublings is

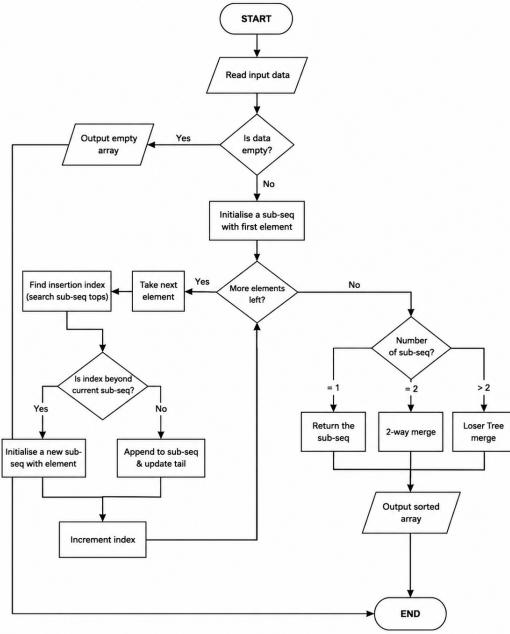


FIGURE 2. Implementation flowchart of LadderSort. The diagram summarises the decomposition and merge control flow, complementing the conceptual overview in Figure 1.

$O(\log K)$. Stage 3 (binary search within the bounded interval of length at most $2s \leq 2K$) costs $O(\log K)$. Hence each placement costs $O(\log K)$ in the worst case, giving Phase 1 cost $O(N \log(K + 1))$.

Phase 2. For $K = 1$, the merge is $O(N)$ (return the single ladder). For $K = 2$, the merge is $O(N)$ (binary merge overhead). For $K > 2$, KWayLoserTree builds in $O(K)$ time and performs N extract-minimum steps each costing $O(\log K)$, for total $O(K + N \log K) = O(N \log K)$ (since $K \leq N$).

Combining both phases: $T(N, K) = O(N \log(K + 1))$. $\square$

For high- K inputs (where K approaches N and LadderSort's overhead exceeds that of a non-adaptive sort), a hybrid variant monitors K during decomposition and falls back to a standard $O(N \log N)$ sort when K exceeds a configurable threshold. This preserves LadderSort's small- K path without penalizing adversarial inputs.

Remark V.1. The bound $T(N, K) = O(N \log(K + 1))$ interpolates correctly: $T(N, 1) = O(N)$ and $T(N, N) = O(N \log N)$. For $K = N^{o(1)}$ (e.g. $K = O((\log N)^c)$), the cost is $o(N \log N)$.

C. AVERAGE-CASE ANALYSIS

For a *uniformly random permutation* of N distinct keys, we bound the expected value of K produced by the LadderSort placement rule.

Proposition V.1 (Expected K under uniform random permutation). *Let A be a uniformly random permutation of*

$\{1, \dots, N\}$. *The expected number of ladders produced by LadderSort satisfies*

$$\mathbb{E}[K] = \Theta(\sqrt{N}).$$

Proof. By Theorem II.1 we have $K = K_{\text{LS}}(A) = \text{LDS}(A)$. Classical results on longest increasing/decreasing subsequences in random permutations (Vershik–Kerov [18], Logan–Shepp [19], and the survey by Aldous–Diaconis [4]) imply that the expected length of the longest decreasing subsequence of a uniform random permutation of $[N]$ is $\Theta(\sqrt{N})$ (in fact $2\sqrt{N} + o(\sqrt{N})$). Hence $\mathbb{E}[K] = \Theta(\sqrt{N})$. $\square$

Corollary V.6 (Average-case complexity). *For a uniformly random permutation of N distinct keys, classical results on longest decreasing subsequences imply $\mathbb{E}[K] = \Theta(\sqrt{N})$. Therefore, under the equality $K = \text{LDS}(A)$ established above,*

$$\mathbb{E}[T(N, K)] = O(N \mathbb{E}[\log K]) = O(N \log \sqrt{N}) = O(N^{\frac{1}{2}} \log N).$$

The constant $\frac{1}{2}$ arises from the $\sqrt{N}$ scaling of K ; one may obtain the sharper $2\sqrt{N} + o(\sqrt{N})$ asymptotic for $\mathbb{E}[K]$ from Vershik–Kerov and Logan–Shepp via Aldous–Diaconis [4], [18], [19].

This asymptotic is consistent with the empirical comparison counts in Section VI: on random inputs LadderSort behaves like an algorithm whose logarithmic factor is about $\frac{1}{2} \log N$ on average.

D. A LOWER BOUND FOR THE K -INTERLEAVING PROMISE

To contextualise LadderSort's $O(N \log(K + 1))$ bound, we give a simple information-theoretic lower bound for the class of inputs that are interleavings of K labeled sorted subsequences.

Proposition V.2 (Worst-case comparison lower bound under K -interleaving). *Consider the set of sequences formed by interleaving K labeled sorted subsequences whose total length is N and whose lengths are balanced (each about N/K). Any comparison-based algorithm that correctly distinguishes all such interleavings must perform at least $\Omega(N \log K)$ comparisons in the worst case.*

Sketch. The number of distinct interleavings of K labeled subsequences of sizes $n_1, \dots, n_K$ is the multinomial coefficient $\frac{N!}{n_1! \dots n_K!}$. For balanced lengths $n_j \approx N/K$ the logarithm of this quantity is $\Theta(N \log K)$ by Stirling's formula, so any decision tree distinguishing these inputs must have depth $\Omega(N \log K)$, implying the stated lower bound on comparisons. $\square$

This lower bound is conditional on the promise that the input is an interleaving of K labeled sorted streams (so the algorithm need only distinguish permutations consistent with that promise). It does not supersede the general $\Omega(N \log N)$ lower bound for arbitrary inputs; rather, it shows that LadderSort's $O(N \log(K + 1))$ behaviour is asymptotically optimal (up to constant factors) for the restricted K -interleaving input class when the streams are balanced.

E. SPACE COMPLEXITY

Lemma V.7 (Space complexity). LadderSort uses $O(N + K)$ auxiliary space.

Proof. Phase 1 storage. Each element a_i is stored in exactly one ladder; the total storage for all ladder elements is N words. The `tops` array uses K additional words. Hence Phase 1 auxiliary space is $O(N + K) = O(N)$ (since $K \leq N$).

Phase 2 storage. For $K \leq 2$, a two-pointer merge uses $O(1)$ extra space beyond the output buffer. For $K > 2$, the loser tree stores K internal nodes and K head pointers, using $O(K)$ extra words beyond the ladders. The output array uses N words.

Total. $O(N + K) = O(N)$ auxiliary words. $\square$

1) Comparison with merge-based sorts

Standard top-down mergesort uses $O(N)$ auxiliary space (a scratch buffer). `std::stable_sort` on most implementations uses $O(N)$ space and falls back to $O(N \log N)$ -compare in-place variants when allocation fails. LadderSort's space bound is comparable to these standard algorithms. In the worst case ($K = N$, descending input), the ladder storage itself is $O(N)$ singletons, but the total space is still $O(N)$.

F. IMPLEMENTATION LOCALITY DISCUSSION

We discuss implementation-level locality considerations without asserting tight cache-oblivious optimality. Phase 1 scans the input sequentially and maintains a compact contiguous tail array `tops[1..K]`; for small K this structure is compact and tends to be efficient in practice. Phase 2 performs K -way merges using per-ladder read buffers and a loser-tree over the active heads; when the number of ladders and buffers fit in available memory, multiway merging reduces the number of passes over the data versus naive repeated two-way merging. Exact I/O bounds and cache-oblivious optimality depend on buffer management choices and hardware parameters M and B , which we did not model or measure precisely in our experiments. Hence we present empirical locality observations in Section VI rather than formal cache-level theorems.

VI. RESULTS AND DISCUSSION

A. EXPERIMENTAL SETUP

This section describes the hardware, compiler, algorithms, and workloads used in our experiments. We report mean measured K for each workload in Table 2.

1) Hardware and Compiler

All experiments ran on a 2020 MacBook Air with an Apple M1 8-core CPU and 8 GB unified memory under macOS 15.6.1. Binaries were compiled with Homebrew GCC 15 using:

```
g++-15 -O3 -march=native -flto -funroll-loops -std=C++20.
```

All runs were single-threaded with no explicit SIMD.

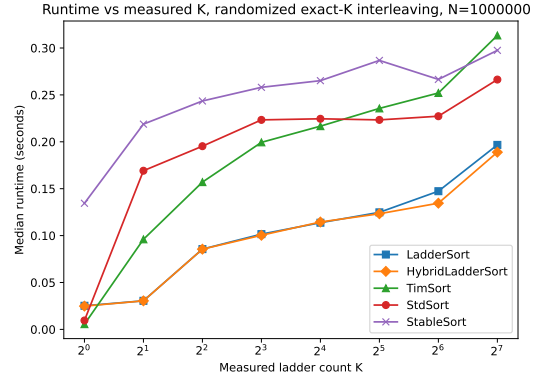


FIGURE 3. Controlled K -sweep at $N = 10^7$. Median runtime increases with K , consistent with the $O(N \log(K + 1))$ bound, and with $O(N \log K)$ behavior for $K \geq 2$. TimSort leads at $K = 1$; LadderSort leads for small $K \geq 2$.

2) Algorithms

- **LadderSort** (raw): two-phase decomposition + adaptive merge.
- **Hybrid LadderSort**: monitors K mid-decomposition; falls back when K exceeds a threshold.
- **TimSort**: `stable` `adaptive` `mergesort` (`gfx::timsort`).
- **QuickSort**: `three-way` `partitioning`, `median-of-three/ninther`.
- **IntroSort**: `std::sort` (`libstdc++`).
- **StableSort**: `std::stable_sort`.
- **MergeSort**: top-down stable mergesort with reusable buffer.

3) Workloads and Measured K

Eight synthetic workloads span the structural range from $K = 1$ to $K = N$. Table 2 reports mean measured K over 10 runs per size. The controlled K -sweep fixes $N = 10^7$ and varies target K using randomized exact- K interleavings, with 10 timed rounds per configuration after one warm-up. Six of 400 rows (1.5%) were filtered for system-level paging outliers (time > 10 s); all correctness checks passed.

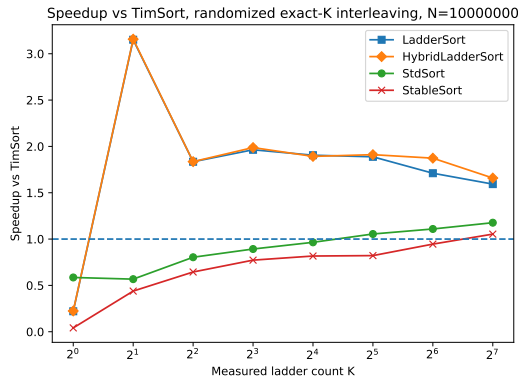
We report mean wall-clock time $\pm$ standard deviation over 10 runs (median for the K -sweep). Correctness is verified after every run.

B. CONTROLLED K -SENSITIVITY SWEEP

Figure 3 shows median runtime vs. K at $N = 10^7$. LadderSort's runtime rises monotonically with K , consistent with the $O(N \log(K + 1))$ bound, and with $O(N \log K)$ behavior for $K \geq 2$. TimSort is fastest at $K = 1$ (single already-sorted run); LadderSort and Hybrid LadderSort lead for $2 \leq K \lesssim 64$, where the loser-tree merge exploits the small fan-in structure. The relative speedup of LadderSort over TimSort across different values of K is presented in Figure 4.

TABLE 2. Mean measured ladder count K over 10 runs. The suite spans $K = 1$ (single sorted run) to $K = N$ (adversarial descending), covering the full structural range of LadderSort’s parameterization.

Workload	$N = 10^6$	$N = 10^7$	$N = 10^8$	Interpretation
Ascending	1.0	1.0	1.0	Single ladder
Two-run riffle	2.0	2.0	2.0	Two interleaved streams
Block-cyclic	8.0	8.0	8.0	Eight producers
Band-limited	14.9	15.9	16.6	Bounded local jitter ($W = 32$)
Partial index	16.0	16.0	16.0	Small segment fan-in
Social-feed	24.0	24.0	24.0	Multi-source fan-in
Random	1985.8	6302.1	19955.1	Weak structure
Descending	10^6	10^7	10^8	Worst case

**FIGURE 4.** Speedup vs. TimSort on the K -sweep. LadderSort and Hybrid lead in the small- K interleaved regime; advantage narrows as K grows.

1) Implementation locality

For small K , the `tops` array is compact (e.g., $K \leq 64$ entries = 256 bytes), which may improve data locality in practice. Similarly, the loser tree layout uses heap-array indexing to improve locality. As K grows, the array size and tree depth increase, which could affect cache behavior. The gradual runtime increase visible in Figure 3 is consistent with this trend, but we did not directly measure cache misses or other hardware counters to confirm the mechanism.

C. WORKLOAD-BY-WORKLOAD ANALYSIS

1) Uniform random ($K \approx \sqrt{N}$)

On random input, `std::sort` (IntroSort) is fastest due to its highly tuned in-place partitioning with minimal memory traffic and strong branch prediction on pivot comparisons. LadderSort is competitive but not leading: Phase 1 placement incurs $O(\log K)$ cost per element with $K \approx \sqrt{N}$, and Phase 2 runs a full K -way merge. The measured comparison counts (≈ 21 per element at $N = 10^6$) match the theoretical expectation of $\frac{1}{2} \log_2 N \approx 10$ for Phase 1 plus $\frac{1}{2} \log_2 N$ for Phase 2, summing to $\approx \log_2 N = 20$. The measured wall-clock runtimes for all algorithms on uniform random inputs are presented in Table 3.

2) Sorted ascending ($K = 1$)

LadderSort is linear ($K = 1$: Phase 1 makes one ladder, Phase 2 returns it), but pays bookkeeping overhead for the initial probe and array allocation. TimSort is $10\times$ faster due to its optimized run-detection path that avoids all comparisons beyond confirming the single ascending run and copies the input directly. This performance gap is not a flaw in LadderSort but reflects that $K = 1$ is TimSort’s absolute best case.

3) Sorted descending ($K = N$)

This is LadderSort’s worst case. Every element starts a new ladder. Phase 1 creates N singletons and Phase 2 runs an N -way merge that consumes $O(N)$ memory for the loser tree. The 3.73 GiB peak RSS at $N = 10^8$ (Table 6) comes from the N singleton ladder pointers and the N -node loser tree. TimSort handles this differently. It detects the descending run in one pass and reverses it in $O(N)$ time using $O(1)$ extra space. LadderSort must create a new ladder for every element. This may produce different branch-prediction behavior compared to TimSort’s reversal loop but we did not measure branch-misprediction rates directly.

4) Band-limited nearly sorted ($K \approx 16$)

LadderSort benefits from the small effective K . Phase 1 places each element using $O(\log 16) = O(4)$ comparisons on average. Phase 2 runs a 16-way merge with a compact loser-tree structure. The compact array and tree layouts may improve data locality compared to larger K values. The runtime per element on this workload is closer to the $K = 16$ regime than the average-case $\sqrt{N}$ regime. StableSort performs similarly because $K = 16$ is moderate enough that a standard merge-sort tree also benefits from the bounded jitter.

5) Block-cyclic interleave ($K = 8, B = 4$)

LadderSort leads across all input sizes. The block-cyclic pattern uses 8 sorted producers each emitting blocks of 4 elements. Phase 1 frequently places consecutive elements into the same ladder. This reduces placement overhead. Phase 2 runs an 8-way merge with a compact loser tree using 8 internal nodes and 8 head pointers. The measured speedup over TimSort at $N = 10^8$ is approximately $1.14\times$. This is consistent with TimSort observing many short runs of length $B = 4$ and paying merge-scheduling overhead for each.

TABLE 3. Runtime (s) on uniform random input: mean $\pm$ std. dev. over 10 runs.

Algorithm	10^6	10^7	10^8	Note
LadderSort	0.0603 (± 0.0003)	0.7115 (± 0.0080)	8.352 (± 0.078)	Competitive
TimSort	0.0761 (± 0.0006)	0.8628 (± 0.0007)	10.021 (± 0.065)	
QuickSort	0.0678 (± 0.0002)	0.7990 (± 0.0006)	9.367 (± 0.133)	
IntroSort	0.0539 (± 0.0005)	0.6242 (± 0.0009)	7.286 (± 0.079)	Fastest
StableSort	0.0612 (± 0.0001)	0.7128 (± 0.0005)	8.398 (± 0.012)	
MergeSort	0.0680 (± 0.0001)	0.7756 (± 0.0017)	9.245 (± 0.059)	

6) Two-run riffle ($K = 2$)

Phase 1 identifies two ladders. Phase 2 then runs a single two-way galloping merge. Galloping is well suited here. Each ladder holds $N/2$ elements in sorted order. Each gallop step skips $\Omega(\log N)$ elements before switching sides. This reduces total comparisons to $O(N)$ with a small constant. LadderSort's advantage over TimSort is approximately $1.35\times$ at $N = 10^8$. TimSort pays overhead from observing $N/2$ short alternating runs and applying its merge-scheduling stack logic on each.

7) Social-feed fan-in ($K = 24$) and partial-index merges ($K = 16$)

Both workloads produce a small fixed K with many duplicate keys. LadderSort's deterministic tie-breaking (Definition II.6) ensures reproducible output on these inputs. This matters for feed-construction and index-maintenance pipelines that require stable ordering within timestamps. Partition-based methods such as `std::sort` degrade on equality plateaus because their three-way partitioning performs poorly when nearly all elements are equal.

D. ABLATION STUDY

Table 4 reports median relative runtime, normalized to LadderFull = 1.00. *NoHint* disables Stage 1 of HintedLowerBound; *NoGallop* uses plain two-pointer merge for $K = 2$; *HeapMerge* replaces the loser tree with a binary heap; *StableMode* enforces global stability via per-element input-position tracking.

1) Implementation considerations for ablation

The $1.51\times$ – $2.05\times$ slowdown from *HeapMerge* is explained by the binary heap's two-child comparison per extract step versus the loser tree's single upward-path pass: a heap on $K = 24$ nodes requires $2\lceil\log_2 24\rceil = 8$ comparisons per extract, while the loser tree requires $\lceil\log_2 24\rceil = 5$ comparisons and a fixed search path (unlike heap sifting, which has data-dependent branching). The *NoHint* slowdown at $K = 24$ ($1.28\times$ at $N = 10^8$) reflects increased search overhead: without the local probe, Stage 2 begins exponential galloping from index 1 every time, which may increase memory access costs. The *StableMode* overhead at $N = 10^8$ ($1.26\times$ for social-feed, $1.43\times$ for two-run riffle) arises from the additional $\log N$ bits per element needed to track original input positions for tie-breaking.

E. HYBRID LADDERSORT

In Table 5, we present the performance of the hybrid variant. The hybrid variant monitors K during Phase 1 and falls back to `std::sort` (IntroSort) when K exceeds a threshold (set to $\lceil\sqrt{N}\rceil$ in our experiments). This preserves LadderSort's small- K performance while avoiding the high-memory overhead of the worst-case $K = N$ scenario.

F. MEMORY AND COMPARISON OVERHEADS

In Table 6 and Table 7, we report peak resident set size (RSS) and median key comparisons per input element at $N = 10^8$ and $N = 10^6$, respectively. The descending workload ($K = N$) shows the worst-case memory footprint for Raw LadderSort, while Hybrid LadderSort avoids this by falling back to `std::sort` (IntroSort). The comparison counts confirm that small- K workloads require only 2–4 comparisons per element for LadderSort, far below general-purpose baselines.

Interpreting these results, the 2.00 comparisons/element on two-run riffle reflects exactly the two Phase 1 placements (one per element confirming $\text{tail} \leq x$) amortized over the galloping merge. The 3.83 on social-feed ($K = 24$) is explained by $\log_2 24 \approx 4.6$ per merge step, offset by the many gallop-skip events where long blocks of equal keys are copied without individual comparisons. The random-input count of 21.24 matches the average-case analysis of Section V-C: $\mathbb{E}[\log_2 K] \approx \frac{1}{2} \log_2 N = 10$ for Phase 1 plus ≈ 10 for Phase 2, plus overhead for the hint mechanism.

G. REAL-WORLD GH ARCHIVE TRACE

Table 8 shows median runtime on a 55,364-record trace from the GitHub Archive, which contains events from 12,783 repositories. The measured $K = 368$ is far below the source count, confirming that the public event stream exhibits low- K structure.

Although $K = 368$ is far below the 12,783 source count, Raw LadderSort is $1.45\times$ faster than TimSort on this trace. The dataset is small enough ($N \approx 55K$) that `std::sort` and `std::stable_sort` benefit from cache-resident in-place sorting with minimal allocation overhead. The primary result is *credibility*: a public multi-source event stream exhibits the low- K noncontiguous structure that LadderSort targets, and Raw LadderSort is competitive against run-adaptive baselines at this scale.

TABLE 4. Ablation results normalized to LadderFull = 1.00. Values > 1 are slower. Hints and loser-tree merging matter most in the $K > 2$ social-feed regime; galloping and hints show mixed results for $K = 2$.

Variant	Purpose	Social-feed ($K = 24$)			Two-run riffle ($K = 2$)		
		10^6	10^7	10^8	10^6	10^7	10^8
LadderFull	Full implementation	1.00	1.00	1.00	1.00	1.00	1.00
NoHint	Disable hinted placement	1.71	1.75	1.28	0.86	0.84	0.68
NoGallop	Disable galloping merge	0.97	0.99	0.72	1.07	1.06	0.79
HeapMerge	Heap instead of loser tree	1.94	2.05	1.51	1.02	1.00	0.82
StableMode	Enforce global stability	1.03	1.07	1.26	1.05	1.07	1.43

TABLE 5. Hybrid LadderSort: fallback rate, speedup over Raw, and speedup over TimSort. Hybrid falls back on high- K inputs, preserving small- K path.

Dataset	N	Fallback	vs. Raw	vs. TimSort	Interpretation
Descending	10^6	1.00	5.71	0.07	Avoids worst case
Descending	10^7	1.00	6.36	0.07	Avoids worst case
Descending	10^8	1.00	4.14	0.03	Avoids worst case
Random	10^6	1.00	1.62	2.51	Fallback improves over raw
Random	10^7	1.00	1.67	2.66	Fallback improves over raw
Random	10^8	1.00	1.75	2.77	Fallback improves over raw
Block-cyclic	10^6	0.00	0.90	1.81	Preserves small- K path
Block-cyclic	10^7	0.00	1.03	1.75	Preserves small- K path
Block-cyclic	10^8	0.00	1.12	1.66	Preserves small- K path

TABLE 6. Peak resident set size at $N = 10^8$ (GiB). Raw LadderSort's descending footprint of 3.73 GiB reflects $K = N$ ladder storage; Hybrid avoids this by falling back.

Dataset	Raw LS	Hybrid LS	TimSort	<code>std::sort</code>	<code>std::stable_sort</code>
Descending	3.73	1.12	1.62	1.62	1.62
Random	1.06	1.26	1.12	0.75	0.90
Social-feed	1.82	1.82	0.78	0.75	1.12

TABLE 7. Median key comparisons per input element at $N = 10^6$ (5 seeds). Small- K workloads require 2–4 comparisons per element for LadderSort, far below general-purpose baselines.

Dataset	K	Raw LS	Hybrid LS	TimSort	<code>std::sort</code>	<code>std::stable_sort</code>
Two-run riffle	2	2.00	2.00	4.93	20.89	18.96
Block-cyclic	8	4.03	4.03	6.07	23.64	25.59
Social-feed	24	3.83	3.83	6.16	17.01	18.95
Random	1985	21.24	27.19	18.63	22.26	29.90
Descending	N	22.95	3.00	1.00	3.00	37.02

H. ASYMPTOTIC SUMMARY

In Table 9, we summarize LadderSort's parametric bounds as functions of input size N and measured ladder count K . The bounds interpolate between $\Theta(N)$ for $K = 1$ (sorted input) and $\Theta(N \log N)$ for $K = N$ (descending input), with a tight $\Theta(N \log K)$ bound for intermediate $K \geq 2$. The space usage is linear in $N + K$, reflecting the auxiliary storage for ladders and the loser tree.

VII. LIMITATIONS AND THREATS TO VALIDITY

A. ALGORITHMIC LIMITATIONS

LadderSort's performance depends critically on K . For adversarial inputs with $K \approx N$ (strictly descending sequences), the algorithm degrades to $\Theta(N \log N)$ with a larger constant factor than highly tuned in-place sorts, because it must ma-

terialize N singleton ladders and then run an N -way merge. LadderSort is not in-place: it requires $O(N)$ auxiliary space for the ladder buffers, which can be prohibitive in memory-constrained environments.

Under the placement and tie conventions used in this paper, Theorem II.1 shows that the Phase 1 ladder count satisfies $K_{LS}(A) = K^*(A) = LDS(A)$, so the measured K is an offline-optimal certificate of the input's decomposability into monotone subsequences. Nevertheless, LadderSort remains a two-phase algorithm: it materializes the ladders (requiring $O(N)$ auxiliary storage in the worst case) and then performs an adaptive merge. Thus the method is not in-place and degrades to $\Theta(N \log N)$ cost on adversarial inputs with $K = \Theta(N)$.

TABLE 8. GH Archive trace: 55,364 records from 12,783 repositories, measured $K = 368$ ($K/N \approx 0.007$), confirming low- K structure in a public event stream.

Algorithm	Median time (s)	K	Fallback	Speedup vs. TimSort	Note
Raw LadderSort	0.002026	368	0.00	1.45	Exposes small- K structure
Hybrid LadderSort	0.001600	237	1.00	1.84	Conservative fallback
TimSort	0.002939	368	0.00	1.00	Run-adaptive baseline
<code>std::sort</code>	0.001001	368	0.00	2.94	Fast standard baseline
<code>std::stable_sort</code>	0.000688	368	0.00	4.27	Fast stable at this scale

TABLE 9. Parametric time and space bounds for LadderSort as functions of input size N and measured ladder count K .

Bound	Expression	Condition	Interpretation
O	$O(N \log(K+1) + K)$	All $K \geq 1$	Phase 1 + loser-tree build
Θ	$\Theta(N)$	$K = 1$	Linear on sorted input
Θ	$\Theta(N \log K)$	$K \geq 2$	Tight for $K \geq 2$
Ω	$\Omega(N)$	All K	Must read N elements
o	$o(N \log N)$	$K = N^{o(1)}$	Sub- $N \log N$ for slow-growing K
Θ	$\Theta(N \log \sqrt{N})$	$K = \Theta(\sqrt{N})$	Random-permutation case
Space	$O(N + K)$	All K	Linear auxiliary

B. WORKLOAD REPRESENTATIVENESS

The synthetic generators cover common systems patterns but not all real workloads. The benchmarks use 32-bit integer keys. Behavior may differ with strings, composite keys or custom comparators due to larger key sizes and pointer indirection. The real-world trace covers one eight-hour window of GitHub events. Other event streams may show different K distributions.

C. IMPLEMENTATION COMPARABILITY

The QuickSort and MergeSort baselines are solid reference implementations. Highly engineered library versions may perform differently. The TimSort baseline uses one specific header-only implementation. Other variants with different run-length bounds or gallop thresholds may produce different results. The behavior of `std::sort` and `std::stable_sort` depends on the standard library and allocator. All results here reflect libstdc++ on ARM64.

D. THREATS TO MEASUREMENT VALIDITY

The evaluation measures wall-clock time, peak RSS and comparison counts. Microarchitectural counters such as cache misses, branch mispredictions and memory bandwidth are not directly measured. The hardware-level interpretations in Section VI are based on known properties of the Apple M1 and are supported by the ablation study. They are not confirmed by direct hardware-counter evidence. Future work will use Performance Monitor Units or PAPI measurements to support these interpretations more directly.

VIII. CONCLUSIONS AND FUTURE WORK

This paper introduced *LadderSort*, a deterministic adaptive sorting algorithm whose running time is parameterized by the ladder count $K := K_{LS}(A)$. LadderSort demonstrates that the minimum number of nondecreasing subsequences provides

a practical and theoretically sound measure of presortedness for adaptive sorting. We established that LadderSort runs in $O(N \log(K+1))$ time, interpolating smoothly between $\Theta(N)$ and $\Theta(N \log N)$ as the degree of disorder increases. We proved the correctness of the algorithm (Theorem V.4), showed that its greedy decomposition phase achieves the offline optimum $K^*(A)$ (Theorem II.1), established its $O(N+K)$ auxiliary-space complexity (Lemma V.7), and derived a rigorous average-case bound of $\Theta(N \log \sqrt{N})$ for uniformly random permutations (Corollary V.6, building on Proposition V.1).

Experimental results show that LadderSort is specially effective for those inputs that are prepared by combining some locally organized streams, such as social-feeds, posting-list merges, telemetry, and inputs produced in related data-processing pipelines. Specifically, it shows a competitive performance on random inputs, whereas in adversarial descending cases, it exhibits a traditional performance of $O(N \log N)$. Without changing the basic algorithmic framework, the hybrid LadderSort exhibits reliability for greater- K inputs.

Future work will explore in-place and parallel variants design of LadderSort, including dynamic selection of fallback threshold of LadderSort through adaptive mechanisms, and analysis on different data types, hardware architectures, implementation settings. Working in these directions may improve the practical usability of LadderSort, while preserving the theoretical foundations.

...

REFERENCES

- [1] N. Auger, V. Jugé, C. Nicaud, and C. Pivoteau, "On the worst-case complexity of TimSort," in *Proceedings of the 26th Annual European Symposium on Algorithms (ESA 2018)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 112. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018, pp. 4:1–4:13.
- [2] J. I. Munro and S. Wild, "Nearly-optimal mergesorts: Fast, practical sorting

- methods that optimally adapt to existing runs,” in *Proceedings of the 26th Annual European Symposium on Algorithms (ESA 2018)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 112. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018, pp. 63:1–63:16.
- [3] M. L. Fredman, “On computing the length of longest increasing subsequences,” *Discrete Mathematics*, vol. 11, no. 1, pp. 29–35, 1975.
 - [4] D. Aldous and P. Diaconis, “Longest increasing subsequences: From patience sorting to the Baik–Deift–Johansson theorem,” *Bulletin of the American Mathematical Society*, vol. 36, no. 4, pp. 413–432, 1999.
 - [5] H. Mannila, “Measures of presortedness and optimal sorting algorithms,” *IEEE Transactions on Computers*, vol. C-34, no. 4, pp. 318–325, 1985.
 - [6] V. Estivill-Castro and D. Wood, “A survey of adaptive sorting algorithms,” *ACM Computing Surveys*, vol. 24, no. 4, pp. 441–476, 1992.
 - [7] R. P. Dilworth, “A decomposition theorem for partially ordered sets,” *Annals of Mathematics*, vol. 51, no. 1, pp. 161–166, 1950.
 - [8] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998.
 - [9] W. Cawley Gelling, M. E. Nebel, B. Smith, and S. Wild, “Multiway PowerSort,” in *Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX 2023)*. SIAM, 2023, pp. 190–200.
 - [10] S. S. Skiena, “Encroaching lists as a measure of presortedness,” *BIT Numerical Mathematics*, vol. 28, no. 4, pp. 775–784, 1988.
 - [11] D. R. Musser, “Introspective sorting and selection algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
 - [12] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
 - [13] S. Edelkamp and A. Weiß, “BlockQuicksort: Avoiding branch mispredictions in quicksort,” in *Proceedings of the 24th Annual European Symposium on Algorithms (ESA 2016)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 57. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2016, pp. 38:1–38:16.
 - [14] E. W. Dijkstra, “Smoothsort, an alternative for sorting in situ,” *Science of Computer Programming*, vol. 1, no. 3, pp. 223–233, 1982.
 - [15] A. Aggarwal and J. S. Vitter, “The input/output complexity of sorting and related problems,” *Communications of the ACM*, vol. 31, no. 9, pp. 1116–1127, 1988.
 - [16] M. Polyntsov, V. D. Grigorev, K. Smirnov, and G. A. Chernishev, “Implementing the comparison-based external sort,” in *New Trends in Database and Information Systems*, ser. Communications in Computer and Information Science. Cham: Springer, 2022, vol. 1652, pp. 500–511.
 - [17] E. Ghasemi, V. Jugé, G. Khalighinejad, and H. Yazdanyar, “Gallop in fast-growth natural merge sorts,” *Algorithmica*, vol. 87, no. 2, pp. 242–291, 2024.
 - [18] A. M. Vershik and S. V. Kerov, “Asymptotics of the Plancherel measure of the symmetric group and the limiting form of young tableaux,” *Soviet Mathematics Doklady*, vol. 18, pp. 527–531, 1977.
 - [19] B. F. Logan and L. A. Shepp, “A variational problem for random young tableaux,” *Advances in Mathematics*, vol. 26, no. 2, pp. 206–222, 1977.